

SQL Tuning

Functional Dependencies

- Functional dependencies are denoted by ρ
- Given: $X \rightarrow Y$.
If two tuples of r (a table with relation schema R) agree on their X -values, then they agree on their Y -values.
- Note: SQL query cannot be used in CHECK constraints
- Functional dependencies may be enforced when LHS has PRIMARY KEY or UNIQUE constraints.
- **Trivial** FD: $X \rightarrow Y$ where $Y \subseteq X$
- **Non-trivial** FD: $X \rightarrow Y$ where $Y \subsetneq X$
- **Completely non-trivial** FD: $X \rightarrow Y$ where $Y \neq \emptyset$ and $Y \cap X = \emptyset$
- Course convention allows: $\emptyset \rightarrow \emptyset, \emptyset \rightarrow Y, X \rightarrow \emptyset$
- **Superkey**: X is a superkey if $X \rightarrow R$ (i.e. X functionally determines all attributes in R)
- **Candidate key**: X is a candidate key if X is a superkey and there is no proper subset of X that is a superkey. i.e. $X \rightarrow R$ and $\nexists Y \subsetneq X$ such that $Y \rightarrow R$, X is minimal.
- **Logical Entailment**: $F \models f$ if every relation that satisfies F also satisfies f . E.g., if $F = \{X \rightarrow Y, Y \rightarrow Z\}$, then $F \models X \rightarrow Z$.

Armstrong's Axioms

- **Reflexivity**: If $Y \subseteq X$, then $X \rightarrow Y$ (i.e. trivial FDs are always valid)
- **Augmentation**: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z (You can add the same attributes to both sides of an FD and it still holds)
- **Transitivity**: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ (i.e. You can short circuit the arrows)
- **Union Rule**: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
- **Decomposition Rule**: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
- **Pseudotransitivity**: If $X \rightarrow Y$ and $YW \rightarrow Z$, then $XW \rightarrow Z$
- **Sound**: If an FD can be derived from Armstrong's Axioms, then it is logically entailed by the set of FDs.
- **Complete**: If an FD is logically entailed by a set of FDs, then it can be derived from Armstrong's Axioms.

Closure of FDs and Attributes

- The closure of a set of functional dependencies Σ (denoted Σ^+) is the set of all FDs that can be logically entailed by Σ .
- $\Sigma^+ = \{X \rightarrow Y \mid \Sigma \models X \rightarrow Y\}$
- **Attribute Closure**: The closure of a set of attributes X under a set of FDs Σ (denoted X^+) is the set of all attributes that are functionally determined by X under Σ .
- $X^+ = \{A \mid \exists (X \rightarrow \{A\}) \in \Sigma^+\}$
- Algorithm to compute **Attribute Closure**:
 1. Initialize $X^+ = X$
 2. For each FD $Y \rightarrow Z$ in Σ , if $Y \subseteq X^+$, then add Z to X^+
 3. Repeat step 2 until no more attributes can be added to X^+

- **Equivalence of FD sets**: Two sets of FDs Σ and Σ' are equivalent if $\Sigma^+ = \Sigma'^+$ (i.e. they have the same closure).
- To check for equivalence, check if they cover each other:
 1. For each FD in Σ_A , use FDs in Σ_B to compute the attribute closure of the LHS of the FD.
 2. If the closure contains the RHS, then the FD is implied by Σ_B .
 3. If all FDs in Σ_A are implied by Σ_B , then Σ_A is implied by Σ_B .
 4. Repeat the same process to check if Σ_B is implied by Σ_A . If both are implied, then Σ_A and Σ_B are equivalent.

Minimal Cover

- A set of FDs Σ is a minimal if:
 1. The **RHS** of every FD in Σ is a single attribute (i.e. FDs are in canonical form)
 2. The **LHS** of every FD in Σ is minimal (i.e. there are no extraneous attributes in the LHS)
 3. There are **no redundant** FDs in Σ (i.e. there is no FD in Σ that can be removed without changing the closure of Σ)
- Algorithm to compute minimal cover:
 1. Decompose FDs so that the RHS of every FD is a single attribute (i.e. put FDs in canonical form)
 2. For each FD, check if there are extraneous attributes in the LHS. If there are, remove them. To check if an attribute A in the LHS of an FD $X \rightarrow Y$ is extraneous, compute the attribute closure of $X - A$ under the set of FDs (including $X \rightarrow Y$). If the closure contains Y , then A is extraneous and can be removed from the LHS.
 3. For each FD, check if it is redundant. To check if an FD $X \rightarrow Y$ is redundant, compute the closure of X under the set of FDs without $X \rightarrow Y$. If the closure contains Y , then $X \rightarrow Y$ is redundant and can be removed from the set of FDs.
- Minimal cover is not unique. There can be multiple minimal covers for a given set of FDs.
- **Compact**: There is no FD with the same LHS.
- **Canonical Cover**: Compact and minimal cover.

The Chase Algorithm

- Test whether a give FD f is logically entailed by a set of FDs Σ .
- Construct a tableau with two rows and n columns, where n is the number of attributes in the relation schema.
- The first row is filled with α and the second row filled with distinct elements.
- For the FD $f : X \rightarrow Y$, replace the elements in the second row with α for all attributes in X .
- Apply the FDs in Σ to the tableau by changing distinct elements into α until no more changes can be made. Note: LHS column α .

Normalization

- **Prime Attribute**: An attribute that is part of a candidate key.
- **1NF**: A relation is in 1NF if all attributes are atomic (i.e. indivisible). There are no repeating groups or arrays in the relation.

- **2NF**: A relation is in 2NF if it is in 1NF and there are no partial dependencies (i.e. no non-prime attribute is functionally dependent on a proper subset of any candidate key). $X \rightarrow \{A\}$ is trivial or A is a prime attribute.
- **3NF**: A relation is in 3NF if it is in 2NF and there are no transitive dependencies ($X \rightarrow \{A\}$ is trivial or A is a prime attribute or X is a superkey).
- **BCNF**: A relation is in BCNF if for every non-trivial FD $X \rightarrow Y$, X is a superkey. Note: BCNF is a stronger condition than 3NF.
- **4NF**: A relation is in 4NF if for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X is a superkey. Note: 4NF is a stronger condition than BCNF.

Decomposition

- **Losslessness**: Reconstructability. A decomposition is lossless-join decomposition if the **natural join** of all the fragments is equivalent to the original relation. (i.e. no spurious tuples are generated when we join the fragments)
- **Dependency Preservation**: A decomposition is dependency preserving if the **union of the projections of the FDs** onto the fragments is equivalent to the original set of FDs (i.e. we can enforce all the FDs by enforcing them on the fragments).
- **Lemma 1, Binary Decomposition**: A decomposition of R into R_1 and R_2 is lossless if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$ (i.e. the common attributes of the two fragments functionally determine one of the fragments).
- **Lemma 2, General Decomposition**: A decomposition of R into $R_1, R_2, \dots, R_n$ is lossless if there exists at least one sequence of binary lossless-join decomposition that generates the decomposition.

Chase Algorithm

- To check if a decomposition is lossless.
- **Input**: A relation schema R , a set of FDs Σ , and a decomposition of R into $R_1, R_2, \dots, R_n$.
 1. Create a table r with schema R with n tuples, where n is the number of fragments in the decomposition.
 2. For each attribute A in R_i , set the A values in the i^{th} tuple as α , treat the rest as distinct symbols.
 3. For each FD $X \rightarrow Y$ in Σ , if the X values in a tuple are the same, then set the Y values in that tuple to be the same. Same α only.
 4. Chase to find a row of all α 's.

Projected Set of FDs

- Way to compute the projected set of FDs on a fragment R_i given the original set of FDs Σ .
 1. Given attribute C is missing
 2. If C appears in the **RHS** of an FD, remove that FD.
 3. If C appears in the LHS of an FD, remove C from the LHS of that FD.
 4. Care for transitivity: If C is in the LHS of an FD and there is another FD where C is in the RHS, we need to check if the transitive dependency can be preserved. If it cannot be preserved, we need to add a new FD with the transitive dependency.
 - 5.

Decomposition Algorithm

- **BCNF Decomposition Algorithm**
 1. **Initialise**: Start with the set of relations $D = \{R\}$.
 2. **Check**: While there is a relation $R_i \in D$ not in BCNF:
 - Find a violation $X \rightarrow Y$ (where $X \rightarrow Y \in \Sigma^+$, $Y \not\subseteq X$, and X is not a superkey for R_i).
 - Compute the attribute closure X^+ with respect to Σ .
 - **Split** R_i into two new relations:
 - 2.1. $R_{i1} = X^+$
 - 2.2. $R_{i2} = (R_i - X^+) \cup X$
 - Replace R_i in D with R_{i1} and R_{i2} .
 3. **Result**: D is a lossless-join decomposition of R into BCNF.
- **Note**: The BCNF decomposition algorithm does not guarantee dependency preservation.
- **3NF Synthesis Algorithm**
 1. **Candidate keys**: Identify candidate keys of R using attribute closure.
 2. **Canonical Cover**: Compute the minimal cover Σ^- of Σ , and compact them.
 3. **Group**: For each $X \rightarrow Y \in \Sigma^-$, create a relation $R_i = X \cup Y$.
 4. **Merge**: If two relations R_1, R_2 have keys X, Z such that $X \rightarrow Z$ and $Z \rightarrow X$, merge them.
 5. **Key Check**: If no R_i contains a candidate key of R , add a new relation R_{key} consisting of any candidate key of R .
 6. **Purge**: Remove any relation R_i that is a subset of another relation R_j .
- **Note**: The 3NF synthesis algorithm guarantees both *lossless-join* and *dependency preservation*. May find BCNF.
- $R = \{A, B, D, E\}$ with $\Sigma = \{BE \rightarrow AD, D \rightarrow E\}$.
Use another table to enforce $D \rightarrow E$ via foreign key.